MECG15603/MCCG15603/UECS3153 Statistical Learning Topic 2b: Supervised Learning: Naive Bayes & LDA

Dr Liew How Hui

Outline

1. Methods of Classification: Generative Models
   - Naive Bayes Classifiers
   - Gaussian NB
   - Categorical NB
   - Multinomial NB and Its Variations
   - Linear Discriminant Analysis
2. Results Interpretation
3. Models Comparison
4. Case Study
5. Lab Practice on Classification Method

Class Arrangement

- Week 9 (Completed): Logistic Regression Model
- Week 10: Lecture 1–3pm (Naive Bayes Model). Practical 3–4pm
- Week 11: Lecture 1–3pm. Practical 3–4pm
- Week 12: Oral Presentation in Lecture 1–3pm. Practical 3–4pm

Generative Models

They share a common mathematical expression:

$$\mathbb{P}(Y = y|\mathbf{X} = \mathbf{x}) = \frac{\mathbb{P}(\mathbf{X} = \mathbf{x}|Y = y)\mathbb{P}(Y = y)}{\mathbb{P}(\mathbf{X} = \mathbf{x})} \tag{1}$$

are based on the Bayes Theorem from probalility theory:

$$\mathbb{P}(B|A) = \frac{\mathbb{P}(A|B)\mathbb{P}(B)}{\mathbb{P}(A)}.$$

Here: $\mathbf{x}$ are inputs/features; y is the output/response. Note: '$\mathbb{P}$' is regarded as probability "mass" and "density" function when the variables are discrete and continuous respectively.

Generative Models (cont)

When the response variable Y is categorical and has K distinct values 1, ..., K, the generative model (1) becomes

$$\mathbb{P}(Y = k|\mathbf{X} = \mathbf{x}) = \frac{\mathbb{P}(\mathbf{X} = \mathbf{x}|Y = k)\mathbb{P}(Y = k)}{\sum_{k=1}^{K} \mathbb{P}(\mathbf{X} = \mathbf{x}|Y = k)\mathbb{P}(Y = k)}, \tag{2}$$

where $k \in \{1, \cdots, K\}$.

Generative Models (cont)

From the generative model (2) for categorical response, we can derive the **generative classifier**

$$\begin{aligned} h_D(\mathbf{x}) &= \underset{k\in\{1,\dots,K\}}{\operatorname{argmax}} \mathbb{P}(Y=k|\mathbf{X}=\mathbf{x}) \\ &= \underset{k\in\{1,\dots,K\}}{\operatorname{argmax}} \frac{\mathbb{P}(\mathbf{X}=\mathbf{x}|Y=k)\mathbb{P}(Y=k)}{\mathbb{P}(\mathbf{X}=\mathbf{x})} \\ &= \underset{k\in\{1,\dots,K\}}{\operatorname{argmax}} \mathbb{P}(\mathbf{X}=\mathbf{x}|Y=k)\mathbb{P}(Y=k) \\ &= \underset{k\in\{1,\dots,K\}}{\operatorname{argmax}} [\ln\mathbb{P}(\mathbf{X}=\mathbf{x}|Y=k)+\ln\mathbb{P}(Y=k)] \end{aligned} \tag{3}$$

Different ways of approximating $\mathbb{P}(\mathbf{X}=\mathbf{x}|Y=k)$ leads to different classifiers.

Outline

Naïve Bayes Classifiers

A **naïve Bayes classifier (NB)** (https://en.wikipedia.org/wiki/Naive_Bayes_classifier) is a generative classifier (3) with **strong independence assumptions** on the likelihood function:

$$
\begin{aligned}
&\mathbb{P}(\mathbf{X} = \mathbf{x}|Y = k)\\
&\mathbb{P}(X_1 = x_1, X_2 = x_2, ..., X_p = x_p|Y = k)\\
&\approx \mathbb{P}(X_1 = x_1|Y = k) \times \cdots \times \mathbb{P}(X_p = x_p|Y = k)\\
&= \prod_{j=1}^{p} \mathbb{P}(X_j = x_j|Y = k).
\end{aligned}
$$

Naïve Bayes Classifiers (cont)

The strong independence assumptions allows the generative classifier (3) to be expressed as

$$
\begin{aligned}
h_D(\mathbf{x}) &= \operatorname*{argmax}_{k\in\{1,\dots,K\}} \mathbb{P}(Y=k)\prod_{j=1}^{p}\mathbb{P}(X_j=x_j|Y=k) \\
&= \operatorname*{argmax}_{k\in\{1,\dots,K\}} \ln\mathbb{P}(Y=k)+\left[\sum_{j=1}^{p}\ln\mathbb{P}(X_j=x_j|Y=k)\right]
\end{aligned} \tag{4}
$$

Naïve Bayes Classifiers (cont)

The prior distribution $\mathbb{P}(Y = k)$ is usually estimated using maximum likelihood estimation (MLE) leading to

$$\widehat{\mathbb{P}(Y = k)} = \frac{\#\{y_i = k\}}{n}. \tag{5}$$

If we know the theoretical distribution of the outcome Y to be uniformly distributed, we can use

$$\mathbb{P}(Y = k) = \frac{1}{K}.$$

Naïve Bayes Classifiers (cont)

The features X_j can either be categorical or be numeric:

- One X_j is numeric — Gaussian NB
- One X_j is categorical — Categorical NB
- All X_j are binary — Bernoulli NB
- All X_j are integral — Multinomial NB & Complement NB(?)

Outline

Gaussian Naïve Bayes

For continuous inputs X_j in (4), it is assume that X_j is 'normal' and satisfies the Gaussian distribution:

$$\mathbb{P}(X_j = x_{ij} | Y = k) = \frac{1}{\sigma_k \sqrt{2\pi}} \exp(-\frac{(x_{ij} - \mu_k)^2}{2\sigma_k^2}). \quad (6)$$

The theoretical estimations of the mean μ_k and the standard deviation σ_k are

$$\mu_k = \mathbb{E}[X_j | Y = k], \quad \sigma_k^2 = \mathbb{E}[(X_j - \mu_k)^2 | Y = k]. \quad (7)$$

Gaussian Naïve Bayes (cont)

The maximum likelihood estimator for (7) provides us the following estimates:

$$\begin{aligned} N_k &= \sum_{i=1}^{n} I(y_i = k) \\ \widehat{\mu}_k &= \frac{1}{N_k} \sum_{i=1}^{n} x_{ij} I(y_i = k), \\ \widehat{\sigma}_k &= \sqrt{\frac{1}{N_k - 1} \sum_{i=1}^{n} (x_{ij} - \widehat{\mu}_{kj})^2 I(y_i = k)}. \end{aligned} \tag{8}$$

Here $I(\cdot)$ turns false to 0 and true to 1, n is number of rows, k is one of the class in $\{1, 2, \cdots, K\}$.

Outline

Categorical NB

If the feature X_j is **categorical** and takes on d_j possible values in $\{c_1^{(j)}, \cdots, c_{d_j}^{(j)}\} =: \mathscr{C}_j$.

The maximum likelihood estimate of the likelihood function is

$$\widehat{\mathbb{P}(X_j = c | Y = k)} = \frac{1}{N_k} \sum_{i=1}^{n} I(x_{ij} = c \bigwedge y_i = k) \qquad (9)$$

for each $c \in \mathscr{C}_i$. Here $N_k = \sum_{i=1}^{n} I(y_i = k)$.

When there is no k such that $x_{ij} = c \wedge y_i = k$, the probability estimate $\widehat{\mathbb{P}(X_j = c | Y = k)} = 0$ and this may be bad for estimation.

Categorical NB (cont)

Laplace smoothing is introduced to prevent zero probabilities in Categorical NB (9):

$$\widehat{\mathbb{P}(X_i = c | Y = j)} = \frac{\sum_{i=1}^{n} I(x_{ki} = c \bigwedge y_k = j) + \alpha}{N_k + \alpha d_j} \quad (10)$$

The constant α is a **smoothing parameter** ranging from 0 to 1.

When $\alpha = 0$, (10) is called **no/without Laplace smoothing**.

When $\alpha = 1$, (10) is called **(with) Laplace smoothing**.

When $0 < \alpha < 1$, (10) is called *Lidstone smoothing*,

Categorical NB (cont)

A combination of Gaussian NB and Categorical NB is usually sufficient for business tabular data which have categorical data columns and numeric data columns.

However, the combination is not sufficient for tabular data with integer entries (e.g. bag of words model for text data).

Statisticians have proposed multinomial NB, complement multinomial NB, Bernoulli NB for text classification based on **bag-of-words** model.

Outline

1. Methods of Classification: Generative Models
   - Naive Bayes Classifiers
   - Gaussian NB
   - Categorical NB
   - Multinomial NB and Its Variations
   - Linear Discriminant Analysis
2. Results Interpretation
3. Models Comparison
4. Case Study
5. Lab Practice on Classification Method

Multinomial NB

The Naïve Bayes algorithm for multinomially distributed data is called a *multinomial Naïve Bayes classifier*:

Application: **text classification**

$$
\begin{aligned}
& h_D(\texttt{document}) \\
=& \operatorname*{argmax}_{k=1,\cdots,K} \mathbb{P}(\texttt{document}|Y=k)\mathbb{P}(Y=k) \\
=& \operatorname*{argmax}_{k=1,\cdots,K} \mathbb{P}(wc_1, wc_2, \cdots, wc_p|Y=k)\mathbb{P}(Y=k)
\end{aligned}
$$

where wc_j is the number of times the word X_j, $j = 1, \cdots, p$, occurred in the `document`, p is the size of the vocabulary.

Multinomial NB (cont)

In text classification, possible entries of "classes" for `document` are "scientific", "economic", "management", etc. A naïve estimate for $\mathbb{P}(Y = j)$ is

$$\mathbb{P}(Y = k) \approx \frac{\text{number of documents of class } k}{\text{number of documents, } n};$$

$$\mathbb{P}(X_j = wc_j | Y = k) \approx \frac{\begin{array}{l}\text{total number of the occurrences of} \\ \text{the word } X_j \text{ in documents of class } k\end{array}}{\text{total number of words } X_1, \cdots, X_p \text{ in documents of class } k} =: \theta_{kj}.$$

Multinomial NB (cont)

A robust estimate of the parameters $\theta_k := (\theta_{k1}, \ldots, \theta_{kp})$ is given by a smoothed version of MLE, i.e. the Laplace smoothing:

$$\mathbb{P}(X_j = wc_j | Y = k) \approx \frac{N_{kj} + \alpha}{N_k + \alpha p}$$

where p is the feature dimension (note data D is assumed to be a $n \times p$ matrix), $N_{kj} = \sum_{y_i=k} wc_j$ is the number of times feature j appears in a sample of class k in the data D, and $N_k = \sum_{j=1}^{p} N_{kj}$ is the total count of all features for class j.

Multinomial NB (cont)

The conditional probability of Multinomial NB is

$$\mathbb{P}(\mathbf{X} = wc|Y = j) = \frac{(\sum_{j=1}^{p} wc_j)!}{wc_1! \times \cdots \times wc_p!} \prod_{j=1}^{p} \theta_{kj}^{wc_j}. \quad (11)$$

In R, libraries `naivebayes` and `fastNaiveBayes` provide multinomial NB.

According to `https://scikit-learn.org/stable/modules/naive_bayes.html`, the Multinomial Naïve Bayes classifiers is implemented as `MultinomialNB`.

```
from sklearn.naive_bayes import MultinomialNB
MultinomialNB(alpha=1.0, fit_prior=True,
  class_prior=None)
```

Complement Multinomial NB

A *complement Naïve Bayes* (CNB) algorithm is an adaptation of the standard MNB algorithm that is particularly suited for imbalanced data sets.

The parameters are estimated by

$$\widehat{\theta}_{kj} = \frac{\alpha_j + \sum_{y_i \neq k} d_{ij}}{\alpha + \sum_{y_i \neq k} \sum_s d_{sj}}$$

where the summations are over all documents i not in class k, d_{ij} is either the count or tf-idf value (term frequency-inverse document frequency, see `https://en.wikipedia.org/wiki/Tf-idf`) of term j in document k.

Complement Multinomial NB (cont)

In Python's Sklearn, CNB is implemented as `ComplementNB` and has the form:

```
from sklearn.naive_bayes import *
ComplementNB(alpha=1.0, fit_prior=True,
  class_prior=None, norm=False)
```

There is no CNB in R because it is inspired by text classification rather than having a firm statistical theory.

Bernoulli NB

Bernoulli Naïve Bayes is used when the data is distributed according to multivariate Bernoulli distributions i.e., echo column x_j is a binary value.

The conditional probability is

$$\mathbb{P}(\mathbf{X} = \mathbf{x} | Y = j) = \prod_{j=1}^{p} \theta_{kj}^{x_j} (1 - \theta_{kj}^{x_j})^{1-x_j} \tag{12}$$

It is implemented in Python as `BernoulliNB`:

```
from sklearn.naive_bayes import *
BernoulliNB(alpha=1.0, binarize=0.0,
  fit_prior=True, class_prior=None)
```

R Implementations

Python is more powerful than R when it comes to text processing. Therefore, Python has the variations of multinomial Naive Bayes available in the `sklearn` library but is a bit weak on the combination of categorical and Gaussian naive Bayes model.

In contrast, R has good supports the combination of categorical and Gaussian naïve Bayes models but only supports the standard multinomial NB.

```
library(naivebayes)
naive_bayes(formula, data, prior = NULL, laplace = 0,
  usekernel = FALSE, usepoisson = FALSE,
  subset, na.action = stats::na.pass, ...)
multinomial_naive_bayes(x, y, prior=NULL, laplace=0.5)
```

Other choices: `e1071::naiveBayes` (slow),

`bnlearn::naive.bayes` (which can only handle categorical data),

`klaR::NaiveBayes` (slow), etc.

Outline

Linear Discriminant Analysis (LDA)

When all inputs are continuous, appart from using Gaussian NB to approximate $\mathbb{P}(\mathbf{X} = \mathbf{x}|Y = y)$, we can also use multinomial Gaussian distribution to approximate it:

$$\begin{aligned} &\mathbb{P}(\mathbf{X} = \mathbf{x}|Y = k) \\ \approx &\frac{1}{\sqrt{\det(\Sigma)\cdot(2\pi)^p}}\exp(-\frac{1}{2}(\mathbf{x}-\mu_k)^T\Sigma(\mathbf{x}-\mu_k)) \end{aligned} \tag{13}$$

where p is the number of columns/features, Σ is the covariance matrix, $\det(\Sigma)$ is the determinant of $p \times p$ matrix Σ, and μ_k is the mean vector (rewritten as column vector) of output $Y = k$ assuming $\mathbf{x}$ is a column vector.

LDA (cont)

Using the **standard moment estimators**, parameters in (13) are estimated as for data (x_{ij}, y_i):

- The prior probability $\mathbb{P}(Y = k) = \frac{\#\{y_i = k\}}{n}$
- The centre of each class $\boldsymbol{\mu}_k$ has the estimation

$$\widehat{\boldsymbol{\mu}}_k = \frac{\sum_{i=1}^n \mathbf{x}_i I(y_i = k)}{\sum_{i=1}^n I(y_i = k)}. \tag{14}$$

- The common covariance matrix $\mathbf{C}$ is:

$$\widehat{\mathbf{C}} = \frac{1}{n-K} \sum_{k=1}^K \sum_{i=1}^n (\mathbf{x}_i - \widehat{\boldsymbol{\mu}}_k)(\mathbf{x}_i - \widehat{\boldsymbol{\mu}}_k)^T I(y_i = k). \tag{15}$$

Outline

Results Interpretation

The generative models are based strongly on the Bayesian statistics philosophy:

$$\text{Posterior} = \frac{\text{Likelihood} \times \text{Prior}}{\text{Average Likelihood}}$$

In the formula (1),

- $\mathbb{P}(Y = y)$ is called the **prior probability**;
- $\mathbb{P}(Y = y|\mathbf{X} = \mathbf{x})$ is called the **posterior probability**, i.e. the 'updated' probability based the input $\mathbf{x}$;
- $\mathbb{P}(\mathbf{X} = \mathbf{x}|Y = y)$ is called the **likelihood function**, which encodes human experience on the distribution of inputs associated to the output y.
- $\mathbb{P}(\mathbf{X} = \mathbf{x})$ is a 'scaling' and is constant w.r.t. to the output.

Results Interpretation (cont)

The probabilistic framework that underlie the generative models is the **Maximum a Posteriori (MAP)**:

$$\begin{aligned}\hat{\theta} &= \text{argmax}_\theta P(\theta|D) \\ &= \underset{\theta}{\text{argmax}} P((\mathbf{x}_y, y_1), \cdots, (\mathbf{x}_n, y_n) \mid \theta) P(\theta).\end{aligned}$$

The MAP obtains a point estimate of an unobserved quantity θ on the basis of empirical data $(\mathbf{x}_i, y_i)$. It is closely related to the method of MLE, but employs an augmented optimisation objective which incorporates a prior distribution over the quantity one wants to estimate. MAP estimation can therefore be seen as a regularisation of MLE.

Results Interpretation (cont)

As a practical user, the simple differences are:

- *Discriminative learning*: We try to approximate $\mathbb{P}(Y|X)$ using
  - function approximation: (multinomial) logistic regression (LR), ANN;
  - data and distance: kNN
  - information, logic and statistics: decision tree, random forest, etc.
- *Generative learning*: We regard $P(Y|X = x)$ as what happens when the prior $P(Y)$ will change with new data $X = x$ is given. This leads to the modelling of the likelihood $P(X|Y)$:
  - Naive Bayes (NB)
  - Discriminative Analysis, e.g. LDA, QDA

Outline

Models Comparison

Since NB does not have any hyperparameters, the model comparison can only be conducted for NB with different subsets of input features based on the performance measurements on the empirical generalisation error:

- cross-validation: when the data size is not large, we can split the data to K blocks and calculate K accuracies (get the average)
- holdout method: split the historical data to training and testing datasets. Compute the accuracies (and/or sensitivity, etc.) and choose a model based on them.

Models Comparison (cont)

In contrast to frequentist statistics's model comparison, in the Bayesian model comparison, prior probabilities are assigned to each of the models, and these probabilities are updated given the data according to Bayes rule.

Given an indexed set of predictive models $M_1, \ldots, M_m$ and associatived prior beliefs in the appropriateness of each model $p(M_i)$, the Bayesian statistics framework uses the model posterior probability

$$p(M_i|D) = \frac{p(D|M_i)p(M_i)}{p(D)}, \quad p(D) = \sum_{i=1}^{m} p(D|M_i)p(M_i)$$

where D is the dataset.

Models Comparison (cont)

When the model M_i is parameterised by θ_i, the model likelihood

$$p(D|M_i)p(M_i) = \int p(D|\theta_i, M_i)p(\theta_i|M_i)d\theta_i.$$

In discrete parameter space, the integral is replaced with summation. Note that the number of parameters $\dim(\theta_i)$ need not be the same for each model.

According to Bayesian statistics, two competing model hypotheses M_i and M_j can be compared using the **Bayes' factor**:

$$\underbrace{\frac{p(M_i|D)}{p(M_j|D)}}_{\text{Posterior odds}} = \underbrace{\frac{p(D|M_i)}{p(D|M_j)}}_{\text{Bayes' factor}} \underbrace{\frac{p(M_i)}{p(M_j)}}_{\text{Prior odds}}.$$

Models Comparison (cont)

For instance, consider a coin tossing problem. We want compare if the coin tossing is biased with M_{biased} and is fair with M_{fair}.

Suppose binomial distribution is used. For $D_1 = 5$ heads & 2 tails,

$$\frac{p(M_{fair}|D_1)}{p(M_{biased}|D_1)} = 1.09$$

Both models are equally OK.

For $D_2 = 50$ heads & 20 tails,

$$\frac{p(M_{fair}|D_2)}{p(M_{biased}|D_2)} = 0.109$$

M_{fair} is only $\approx$11% of the likelihood of M_{biased}.

Models Comparison (cont)

Computer simulation examples are given at
`https://bookdown.org/kevin_davisross/bayesian-reasoning-and-methods/model-comparison.html`

For theory, see
`https://en.wikipedia.org/wiki/Bayes_factor`

Outline

Case Study 1: Categorical NB

Example 1: Table Q3(d) shows a data set containing 7 observations with 3 categorical predictors, X_1, X_2 and X_3.

Observation	X_1	X_2	X_3	Y
1	C	No	0	Positive
2	A	Yes	1	Positive
3	B	Yes	0	Negative
4	B	Yes	0	Negative
5	A	No	1	Positive
6	C	No	1	Negative
7	B	Yes	1	Positive

Table Q3(d)

Without Laplace smoothing, predict the response, Y for an observation with $X_1 = \text{B}$, $X_2 = \text{Yes}$ and $X_3 = 1$ using Naïve Bayes approach. (5 marks)

Case Study 1 (cont)

Solution: Let '$+$' denote 'Positive' and '-' denote 'Negative'.

prior, $\mathbb{P}(Y)$	$\mathbb{P}(X_1\|Y)$	$\mathbb{P}(X_2\|Y)$	$\mathbb{P}(X_3\|Y)$	prop, Π	$\hat{Y}$
$\mathbb{P}(+) = \frac{4}{7}$	$\mathbb{P}(\text{B}\|+) = \frac{1}{4}$	$\mathbb{P}(\text{Yes}\|+) = \frac{1}{2}$	$\mathbb{P}(1\|+) = \frac{3}{4}$	0.0536	
$\mathbb{P}(-) = \frac{3}{7}$	$\mathbb{P}(\text{B}\|-) = \frac{2}{3}$	$\mathbb{P}(\text{Yes}\|-) = \frac{2}{3}$	$\mathbb{P}(1\|-) = \frac{1}{3}$	0.0635	$\checkmark$, $-$

Since $\mathbb{P}(Y = -|X) > \mathbb{P}(Y = +|X)$, Y has higher probability to be "Negative".

Case Study 2: Categorical NB

Example 2: Consider the following case given in `https://machinelearningmastery.com/naive-bayes-tutorial-for-machine-learning/`

Weather	Car	Y
sunny	working	go-out
rainy	broken	go-out
sunny	working	go-out
sunny	working	go-out
sunny	working	go-out
rainy	broken	stay-home
rainy	broken	stay-home
sunny	working	stay-home
sunny	broken	stay-home
rainy	broken	stay-home

Construct the categorical Naïve Bayes model for the above data.

Case Study 2 (cont)

Solution: Let X_1=Weather, X_2=Car. The categorical Naïve Bayes model:

$$\begin{aligned}&\mathbb{P}(Y=j|\mathbf{X}=\mathbf{x})\\ \propto&\mathbb{P}(Y=j)\times\mathbb{P}(X_1=x_1|Y=j)\times\mathbb{P}(X_2=x_2|Y=j)\end{aligned}$$

where Prior, $\mathbb{P}(Y)=\begin{cases}0.5, & Y=out\\ 0.5, & Y=stay\end{cases}$

$\mathbb{P}(X_1|Y=out)=\begin{cases}\frac{4}{5}, & X_1=sunny\\ \frac{1}{5}, & X_1=rainy\end{cases}$,

$\mathbb{P}(X_1|Y=stay)=\begin{cases}\frac{2}{5}, & X_1=sunny\\ \frac{3}{5}, & X_1=rainy\end{cases}$

Case Study 2 (cont)

Solution (cont):

$$\mathbb{P}(X_2|Y = out) = \begin{cases} \frac{4}{5}, & X_2 = working \\ \frac{1}{5}, & X_2 = broken \end{cases},$$

$$\mathbb{P}(X_2|Y = stay) = \begin{cases} \frac{1}{5}, & X_2 = working \\ \frac{4}{5}, & X_2 = broken \end{cases}$$

Case Study 3: Gaussian NB

Example 3: The table below shows the data collected for predicting whether a customer will default on the credit card or not:

customer	balance	student	Default
1	500	No	N
2	1980	Yes	Y
3	60	No	N
4	2810	Yes	Y
5	1400	No	N
6	300	No	N
7	2000	Yes	Y
8	940	No	N
9	1630	No	Y
10	2170	Yes	Y

Case Study 3 (cont)

(a) Compute the probability density of customer with balance 2080, given Default=Y.
(b) Compute the probability of customer who is a student, given Default=Y.
(c) Calculate the "probability density" of default for a student customer with balance 2080 by using the Naïve Bayes assumption.

Case Study 3 (cont)

Note: This question just asks for specific answer without the full model, so we don't need to write the full model.

(a) **Solution**:

$$\mathbb{P}(\texttt{balance} = 2080 \mid \texttt{Default} = Y)$$

$$= \frac{1}{s_Y\sqrt{2\pi}} \exp(-\frac{(2080 - \mu_Y)^2}{2s_Y^2}) = 0.0009162$$

where $\mu_Y = \frac{1980+2810+2000+1630+2170}{5} = 2118$;
$s_Y = 433.7857$

Gaussian Naïve Bayes (cont)

(b) **Solution**: $\mathbb{P}(\texttt{student} = \mathit{Yes} \mid \texttt{Default} = Y) = \dfrac{4}{5}$

(c) **Solution**:

$\mathbb{P}(\texttt{student} = \mathit{Yes} \mid \texttt{Default} = N) = \frac{0}{5} = 0$

$$
\begin{aligned}
&\mathbb{P}(\texttt{Default} = Y \mid \texttt{balance} = 2080,\ \texttt{student} = \mathit{Yes}) \\
=&\frac{\mathbb{P}(\texttt{balance} = 2080,\ \texttt{student} = \mathit{Yes} \mid \texttt{Default} = Y)\mathbb{P}(\texttt{Default} = \texttt{Y})}{\mathbb{P}(\texttt{balance} = 2080,\ \texttt{student} = \mathit{Yes}) =: \mathbb{P}(\ldots)} \\
=&\frac{\mathbb{P}(\texttt{balance} = 2080,\ \texttt{student} = \mathit{Yes} \mid \texttt{Default} = Y)\mathbb{P}(\texttt{Default} = \texttt{Y})}{\mathbb{P}(\ldots \mid \texttt{Default} = Y)\mathbb{P}(\texttt{Default} = \texttt{Y}) + \mathbb{P}(\ldots \mid \texttt{Default} = N)\mathbb{P}(\texttt{Default} = \texttt{N})} \\
=&\frac{0.0009162 \times \frac{4}{5} \times \frac{5}{10}}{0.0009162 \times \frac{4}{5} \times \frac{5}{10} + \mathbb{P}(\texttt{balance} = 2080|\texttt{Default} = N) \times 0 \times \frac{5}{10}} = 1
\end{aligned}
$$

Case Study 3 (cont)

Remark on **Example** 3: Let x_1=balance, x_2=student, y=Default. The Naive Bayes Model is

$$h_D(x_1, x_2) = \operatorname*{argmax}_j \mathbb{P}(x_1|y = j)\mathbb{P}(x_2|y = j)\mathbb{P}(y = j).$$

where the prior $\mathbb{P}(y) = \begin{cases} 0.5 & y = N \\ 0.5 & y = Y \end{cases}$

$$\mathbb{P}(x_2|y = N) = \begin{cases} 1 & x_2 = No \\ 0 & x_2 = Yes \end{cases}$$

$$\mathbb{P}(x_2|y = Y) = \begin{cases} 1/5 & x_2 = No \\ 4/5 & x_2 = Yes \end{cases}$$

Case Study 3 (cont)

Remark on **Example** 3 (cont):

$$\mathbb{P}(x_1|y=N) = \frac{1}{\sqrt{2\pi}(533.6666)} \exp\left\{-\frac{(x_1-640)^2}{2(533.6666)^2}\right\}$$

$$\mathbb{P}(x_1|y=Y) = \frac{1}{\sqrt{2\pi}(433.7857)} \exp\left\{-\frac{(x_1-2118)^2}{2(433.7857)^2}\right\}$$

Case Study 4: Gaussian NB

Example 4:
A more efficient marketing strategy can be achieved by targeting the customers who have higher probability to complete a purchase. Hence, you have been asked to predict whether a customer will buy the product based on their demographic data such as age, race, gender and income. Table Q3(c) shows the data collected from previous records.

Case Study 4 (cont)

Cust.	Age	Race	Gender	Income	Result
1	52	Indian	Male	RM 11,500	Not Buy
2	22	Chinese	Female	RM 6,500	Buy
3	30	Chinese	Male	RM 8,000	Buy
4	26	Malay	Male	RM 8,500	Buy
5	27	Indian	Female	RM 6,500	Buy
6	32	Chinese	Female	RM 9,500	Not Buy
7	33	Indian	Male	RM 4,000	Not Buy
8	50	Malay	Female	RM 10,000	Buy
9	31	Chinese	Female	RM 5,500	Buy
10	27	Malay	Male	RM 9,200	Not Buy

Table Q3(c)

Case Study 4 (cont)

(i) State an assumption used in Naïve Bayes approach. (1 mark)

(ii) Using Naïve Bayes approach without Laplace smoothing, predict whether a Malay female customer, aged 29, with income RM7,800, will buy the product. (9 marks)

Case Study 5: Software Support

Gaussian Naïve Bayes (Classifier) is available in Python as `GaussianNB` of the form:

```
from sklearn.naive_bayes import GaussianNB
GaussianNB(priors=None, var_smoothing=1e-09)
```

All the above mentioned naïve Bayes models are available in R except the complement NB. R provides unified functions such as `naivebayes::naive_bayes`, `e1071::naiveBayes`, `bnlearn::naive.bayes` (which can only handle categorical data), `klaR::NaiveBayes`.

```
naive_bayes(formula, data, prior = NULL, laplace = 0,
  usekernel = FALSE, usepoisson = FALSE,
  subset, na.action = stats::na.pass, ...)
```

Case Study 6: Categorical NB with Laplace Smoothing

An issue faced by a Naïve Bayes classifier with "discrete" data is the numerator in (9) being zero, i.e. $n_{X=c,Y=j} = 0$. In this case, the posterior probability will become zero regardless of the value of other density functions and the Naïve Bayes classifier will fail.

Case Study 6 (cont)

Example 5: By using the data from **Example** 3, perform the following tasks.

(a) Compute the probability density of customer with balance 2080, given Default=N.

(b) Compute the probability of customer who is a student, given Default=N.

(c) Calculate the “probability density” of non-default for a student customer with balance 2080 by using NB model without Laplace smoothing.

(d) Redo part (b) and (c) with Laplace smoothing.

Case Study 6 (cont)

(a) Solution: $\mathbb{P}(\texttt{balance} = 2080 \mid \texttt{Default} = N) = \frac{1}{s_N\sqrt{2\pi}}\exp(-\frac{(2080-\mu_N)^2}{2s_N^2}) = 1.9616 \times 10^{-5}$
where $\mu_N = \frac{500+60+1400+300+940}{5} = 640; \quad s_N = 533.6666$
(b) Solution:
$\mathbb{P}(\texttt{student} = \mathit{Yes} \mid \texttt{Default} = N) = \frac{0}{5} = 0$
(c) Solution:

$$\mathbb{P}(\texttt{Default} = N \mid \texttt{balance} = 2080,\ \texttt{student} = \mathit{Yes})$$
$$= \frac{1.9616 \times 10^{-5} \times 0 \times \frac{5}{10}}{\mathbb{P}(\texttt{balance} = 2080,\ \texttt{student} = \mathit{Yes})} = 0.$$

Case Study 6 (cont)

Remark: This situation is normally happened to the categorical variable. To avoid the stated problem, *Laplace smoothing* or `https://en.wikipedia.org/wiki/Additive_smoothing` is applied to the Naïve Bayes classifier. Laplace smoothing modified the density function of categorical variable by adding α (by default, $\alpha = 1$) to each variable per class:

$$\mathbb{P}(X = x_i | Y = k) = \frac{n_{X=x_i;Y=k} + \alpha}{n_{Y=k} + d\alpha} \tag{16}$$

where d is the number of classes in the categorical variable X.

Case Study 6 (cont)

(d) Redo part (b) and (b) by applying Laplace smoothing:
The "continuous" variable is the same:

$$\mathbb{P}(\texttt{balance} = 2080 \mid \texttt{Default} = N) = 1.9616 \times 10^{-5}$$

The categorical variable needs the Laplace smoothing ($\alpha = 1$, $d = 2$ for $\texttt{student}$=Yes or No)

$$\mathbb{P}(\texttt{student} = \mathit{Yes} \mid \texttt{Default} = N) = \frac{0+1}{5+2}$$

Case Study 6 (cont)

Therefore,

$$
\begin{aligned}
&\mathbb{P}(\texttt{Default} = N \mid \texttt{balance} = 2080,\ \texttt{student} = Yes) \\
=&\frac{\mathbb{P}(2080, Yes \mid N)\mathbb{P}(N)}{\mathbb{P}(2080, Yes \mid N)\mathbb{P}(N) + \mathbb{P}(2080, Yes \mid Y)\mathbb{P}(Y)} \\
=&\frac{1.9616 \times 10^{-5} \times \frac{1}{7} \times \frac{5}{10}}{1.9616 \times 10^{-5} \times \frac{1}{7} \times \frac{5}{10} + 0.000916154 \times \frac{5}{7} \times \frac{5}{10}} \\
=&\frac{1.401151 \times 10^{-6}}{1.401151 \times 10^{-6} + 0.0003271979} = 0.004264014
\end{aligned}
$$

Outline

Lab Practice on Classification Method

`prac_cls2-nb.R` (Naive Bayes)

Make sure you have formed a group, pick a group leader and a dataset of interest and start working on your assignment which requires you to work on at least two supervised learning classification models in the assignment.

You should practice to load the data using R and then perform feature-response analysis and pattern analysis on the data using the knowledge gained from Week 1 to Week 4.